

Google Career — Day 5 Interview Questions & Answers

Day 5 is deliberately focused on PostgreSQL + SQL + persistent backend engineering + database testing + Prefix Sum DSA + system design. The source plan explicitly frames the project transition as FastAPI → DocumentService → DocumentRepository → PostgreSQL, while keeping the API contract stable.

Below is the detailed interview bank I recommend studying after completing the implementation, because many questions are designed around your actual Project 1 architecture.

PART I — POSTGRESQL FUNDAMENTALS

Q1. What is PostgreSQL?

Answer:

PostgreSQL is an open-source relational database management system.

It stores structured data in tables and provides mechanisms for:

- SQL queries
- transactions
- constraints
- indexes
- data integrity

For Project 1, PostgreSQL is appropriate because document metadata such as id, name, document_type, version, status, created_at, and updated_at is structured relational data.

Interview follow-up

Why PostgreSQL instead of an in-memory dictionary?

An in-memory dictionary loses data when the application restarts and does not provide the durability, transactional guarantees, indexing, concurrency control, and query capabilities of a database.

Q2. What is a relational database?

Answer:

A relational database stores data in structured tables consisting of rows and columns.

Relationships between entities can be represented using keys.

The important mental model is:

Database



Schema



Tables



Rows



Columns

This is the relational model relevant to your Day 5 document service.

Q3. What is a table?

Answer:

A table is a collection of related records organized into rows and columns.

For Project 1:

documents

is the table.

Each row represents one document.

Each column represents one property of the document.

Example:

id	name	document_type	version	status
doc-001	engine-requirement	requirement	1.0	created

Q4. What is a row?

Answer:

A row represents one record in a table.

For example:

doc-001	engine-requirement	requirement	1.0	created
---------	--------------------	-------------	-----	---------

represents one document.

Q5. What is a column?

Answer:

A column represents an attribute of every record.

For example:

id
name
document_type
version
status

Each document has a value associated with those attributes.

Q6. What is a primary key?

Answer:

A primary key uniquely identifies each row in a table.

For example:

```
id TEXT PRIMARY KEY
```

For our document service, id identifies a document.

A primary key provides:

- uniqueness
- identity
- efficient lookup
- a target for relationships
- database-level integrity

Q7. Why shouldn't document name be the primary key?

Answer:

Because names may not be unique and may change.

For example:

requirements.pdf
requirements.pdf

could refer to different versions or documents.

A stable identifier is better:

doc-001
doc-002

The name becomes an attribute rather than the identity of the record.

Q8. What is a foreign key?

Answer:

A foreign key is a column that references a key in another table.

For example:

documents

id

tests

document_id

tests.document_id could reference:

documents.id

This allows the database to enforce relationships between entities.

Your Day 5 project does not require implementing a complex foreign-key model yet, but you should understand the concept.

Q9. What is a database constraint?

Answer:

A constraint is a database rule that restricts invalid data.

Examples include:

PRIMARY KEY

NOT NULL

UNIQUE

CHECK

FOREIGN KEY

Constraints move some correctness guarantees from application code into the database.

Q10. What does NOT NULL mean?

Answer:

It means a column must contain a value.

Example:

```
name TEXT NOT NULL
```

The database rejects a row where name is missing.

Q11. What does UNIQUE mean?

Answer:

UNIQUE prevents duplicate values within a column or combination of columns.

Example:

```
name TEXT UNIQUE
```

However, you should not automatically make every business field unique.

The requirement must justify it.

Q12. What is a CHECK constraint?

Answer:

A CHECK constraint ensures that a value satisfies a condition.

For example:

```
CHECK (  
    document_type IN ('requirement', 'design', 'test')  
)
```

This prevents unsupported document types from entering the database.

Q13. Why enforce validation both in the API and database?

Answer:

They protect different boundaries.

API validation

Protects the application boundary:

Client

↓

Pydantic

Database constraints

Protect the persistence boundary:

Application

↓

Database

Defense in depth means that even if another code path bypasses the API validation, the database can still reject invalid data.

PART II — SQL

The Day 5 scope explicitly requires INSERT, SELECT, UPDATE, and DELETE, without jumping into advanced SQL.

Q14. What is SQL?

Answer:

SQL stands for Structured Query Language.

It is used to interact with relational databases.

For example:

```
SELECT *  
FROM documents;
```

Q15. What does INSERT do?

Answer:

INSERT creates a new row.

Conceptually:

```
INSERT INTO documents (...)  
VALUES (...);
```

For Project 1, this corresponds to document creation.

Q16. What does SELECT do?

Answer:

```
SELECT retrieves data.  
  
SELECT *  
FROM documents;
```

It can retrieve:

- all rows
- selected columns
- filtered rows
- joined data
- aggregated data

Q17. What does WHERE do?

Answer:

WHERE filters rows.

```
SELECT *  
FROM documents  
WHERE id = %s;
```

Without the WHERE, the query could return every document.

Q18. What does UPDATE do?

Answer:

```
UPDATE modifies existing rows.
```

Example:

```
UPDATE documents
SET version = %s
WHERE id = %s;
```

The WHERE clause is extremely important.

Q19. What happens if you accidentally omit the WHERE from an UPDATE?

Answer:

Every row may be updated.

For example:

```
UPDATE documents
SET status = 'created';
```

could modify every document.

This is a classic production failure.

Q20. What does DELETE do?

Answer:

```
DELETE removes rows.
DELETE FROM documents
WHERE id = %s;
```

Again, omitting WHERE can delete every row.

Q21. What is the difference between DELETE and DROP?

Answer:

```
DELETE removes rows.
DELETE FROM documents;
```

The table remains.

DROP TABLE removes the table itself.

```
DROP TABLE documents;
```

So:

DELETE → data
DROP → database object

Q22. Why should SQL queries be parameterized?

Answer:

To prevent SQL injection and correctly handle values.

Bad:

```
query = f"SELECT * FROM documents WHERE id = '{document_id}'"
```

Better:

```
query = "SELECT * FROM documents WHERE id = %s"  
cursor.execute(query, (document_id,))
```

The database driver handles the parameter separately from the SQL structure.

Q23. What is SQL injection?

Answer:

SQL injection occurs when untrusted input is incorrectly incorporated into SQL, allowing an attacker to manipulate the query.

For example, dynamically constructing SQL with raw user input can allow malicious SQL fragments to alter the query's meaning.

Parameterized queries are the primary defense for this class of problem.

PART III — TRANSACTIONS

Transactions are explicitly part of Day 5 because you need to understand multiple database operations behaving as one logical unit.

Q24. What is a database transaction?

Answer:

A transaction is a logical unit of database work.

Conceptually:

BEGIN

operation 1

operation 2

COMMIT

If something fails:

ROLLBACK

The goal is to prevent partially completed operations.

Q25. What is COMMIT?

Answer:

COMMIT permanently applies the changes made within a transaction.

BEGIN

↓

INSERT

↓

UPDATE

↓

COMMIT

After the commit, the changes become durable according to the database's guarantees.

Q26. What is ROLLBACK?

Answer:

ROLLBACK cancels changes made within the current transaction.

Example:

BEGIN

↓

INSERT

↓

UPDATE

↓

ERROR

↓

ROLLBACK

The transaction's changes are undone.

Q27. Why are transactions important?

Answer:

They protect consistency when multiple operations must succeed or fail together.

Suppose:

Create document

Create associated metadata

Create audit record

If the first two succeed but the third fails, we may not want the database left in a partially completed state.

A transaction can make the operations atomic.

Q28. What happens if operation 1 succeeds but operation 2 fails?

Answer:

Without appropriate transaction handling, operation 1 may remain committed while operation 2 fails.

That can create inconsistent state.

With a transaction:

operation 1

operation 2

↓

failure

↓

ROLLBACK

the application can undo the transaction's changes.

Q29. Explain ACID.

Answer:

ACID describes important transaction properties:

Atomicity

All operations happen or none happen.

Consistency

A transaction moves the database between valid states.

Isolation

Concurrent transactions should not incorrectly interfere with each other.

Durability

Committed changes survive failures according to the database's durability guarantees.

Q30. What is isolation?

Answer:

Isolation controls how concurrent transactions interact.

For example:

Transaction A

↓

updates document

Transaction B

↓

reads/updates same document

The database needs rules governing what B can observe and how concurrent modifications interact.

This becomes increasingly important when your service has many concurrent API requests.

PART IV — INDEXES

Q31. What is a database index?

Answer:

An index is an additional data structure maintained by the database to make certain queries faster.

For example:

```
CREATE INDEX idx_documents_type
ON documents(document_type);
```

Instead of scanning every row for some queries, PostgreSQL may use the index to locate matching rows more efficiently.

Q32. Why do indexes improve reads?

Answer:

Without a useful index, the database may need to inspect many or all rows.

With an appropriate index, it can often locate matching records more efficiently.

Conceptually:

No index:

```
row 1 → check
row 2 → check
row 3 → check
```

...

```
row N → check
```

An index provides an organized lookup structure.

Q33. Why can indexes hurt writes?

Answer:

Indexes themselves must be maintained.

When a row is:

INSERTED

UPDATED

DELETED

the relevant indexes may also need modification.

Therefore:

More indexes

↓

potentially faster reads

↓

more storage + write/maintenance cost

This trade-off is explicitly part of the Day 5 scope.

Q34. Should you index every column?

Answer:

No.

Indexes have:

- storage cost
- write cost
- maintenance cost
- memory/cache implications

Indexes should support actual query patterns.

For Day 5, learn index fundamentals rather than prematurely designing sophisticated indexing.

Q35. What index would you consider for GET /documents/{document_id}?

Answer:

The primary key on id already provides the appropriate indexing behavior for primary-key lookup.

Therefore we don't need to create another redundant index on id.

Q36. What determines whether an index is useful?

Answer:

The query workload.

You should examine:

- filtering columns
- joins
- sorting
- query frequency
- cardinality/selectivity
- read/write ratio

The correct index depends on how the application actually queries the database.

PART V — DATABASE DESIGN FOR PROJECT 1

Q37. What schema are you designing for Project 1?

Answer:

The Day 5 design uses:

documents

id

name

document_type

version

status

created_at

updated_at

The important engineering principle is to avoid adding fields merely because they might become useful later.

Q38. Why have created_at and updated_at?

Answer:

They provide temporal information.

created_at tells us when the document was created.

updated_at tells us when it was last modified.

This becomes useful for:

- debugging
- auditing
- sorting
- synchronization
- APIs
- monitoring
- future change tracking

Q39. Why store status?

Answer:

status represents the current state of the document.

For example:

created

processing

validated

failed

The exact states should reflect actual project requirements rather than hypothetical future functionality.

Q40. Why is document_type important?

Answer:

It identifies the semantic category of the document.

For Project 1, examples include:

requirement

design

test

It can also support filtering and future retrieval workflows.

Q41. Where should document business validation happen?

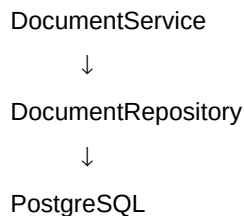
Answer:

Primarily in the service/domain layer, while the database should enforce persistence-level invariants.

The architecture remains:

FastAPI

↓



The service owns business behavior; the repository owns persistence.

PART VI — REPOSITORY ARCHITECTURE

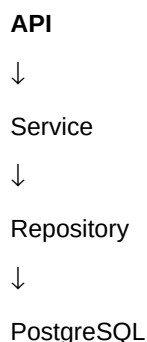
This is one of the highest-value areas for your Google interview, because Day 5 is specifically about replacing the persistence implementation without redesigning the application.

Q42. Why put PostgreSQL behind a repository?

Answer:

The repository isolates persistence details from business logic.

Architecture:



The service doesn't need to know whether data is stored in:

memory
PostgreSQL
another database

This makes the persistence implementation replaceable and keeps responsibilities separated.

The Day 5 objective explicitly calls for making PostgreSQL a persistence implementation behind the existing repository abstraction.

Q43. Why shouldn't DocumentService execute SQL directly?

Answer:

Because that mixes business logic and persistence logic.

Bad:

DocumentService

■■■ validate document

■■■ execute SQL

■■■ manage cursor

■■■ map database rows

■■■ enforce business rules

Better:

DocumentService

■■■ business logic

DocumentRepository

■■■ database operations

This improves:

- separation of concerns
- testing
- maintainability
- replaceability
- readability

Q44. What should the service layer do?

Answer:

The service layer should own business behavior.

For example:

validate business rules

coordinate operations

decide whether an operation is valid

call repository

translate domain-level outcomes

It should not become a thin wrapper around raw SQL.

Q45. What should the repository layer do?

Answer:

The repository should handle persistence operations.

For example:

create(document)

```
list()
get_by_id(id)
update(...)
delete(id)
```

Internally these methods may execute SQL.

The rest of the application should not need to know those SQL details.

Q46. What happens if you later replace PostgreSQL with another persistence system?

Answer:

Ideally the API and service layers remain largely unchanged.

Only the persistence implementation changes.

Conceptually:

```

      ■■■ PostgreSQLRepository
DocumentService ■■■■■■■■■ InMemoryRepository
                  ■■■ OtherRepository
```

This is the value of the abstraction.

Q47. Why is this architecture useful for testing?

Answer:

It allows layers to be tested independently.

For example:

API tests

Service tests

Repository/database tests

You can test business logic without necessarily requiring a real database for every unit test.

However, integration tests should still verify the actual PostgreSQL implementation.

PART VII — DATABASE CONNECTION MANAGEMENT

Q48. How does a Python application connect to PostgreSQL?

Answer:

Typically through a PostgreSQL-compatible database driver.

The application provides connection information such as:

host

port

database

username

password

or a database URL.

The repository/database layer uses that connection to execute SQL.

Q49. What is a connection pool?

Answer:

A connection pool maintains a collection of reusable database connections.

Instead of:

request

↓

open new connection

↓

query

↓

close connection

for every request, the application can reuse connections:

connection pool

■■■ connection 1

■■■ connection 2

■■■ connection 3

■■■ ...

This reduces connection establishment overhead and controls concurrency.

Q50. Why shouldn't every API request create a completely new database connection?

Answer:

Creating connections has overhead.

At high request rates, repeatedly opening connections can:

- increase latency

- consume database resources
- exhaust connection limits
- reduce throughput

Connection pooling provides controlled reuse.

Q51. What happens if PostgreSQL becomes unavailable?

Answer:

The repository operation fails.

The application should:

- 1. detect the database failure,
- 2. log the error,
- 3. avoid exposing sensitive internals,
- 4. return an appropriate server-side response,
- 5. potentially retry only where safe,
- 6. maintain observability.

The important point is not to pretend the operation succeeded.

PART VIII — CONFIGURATION AND SECRETS

Q52. Why use environment variables for database configuration?

Answer:

Because configuration should not be hard-coded into source code.

For example:

`DATABASE_URL`

can differ between:

development

testing

staging

production

This also prevents credentials from being committed into Git.

Q53. Should .env be committed?

Answer:

No, if it contains secrets.

Instead:

.env

should normally be ignored, while:

.env.example

can document the required configuration without containing real credentials.

Q54. What should happen if DATABASE_URL is missing?

Answer:

The application should fail clearly during configuration/startup rather than silently falling back to an unintended database.

Configuration errors should be explicit.

PART IX — DATABASE TESTING

Q55. How do you test a PostgreSQL-backed API?

Answer:

Test the API through its HTTP interface while using a controlled test database.

For example:

POST /documents



FastAPI



Service



Repository



Test PostgreSQL

Then verify the resulting database-backed behavior.

Q56. What should you test for document creation?

Answer:

Send:

POST /documents

and verify:

- HTTP status
- response schema
- generated ID
- stored values
- database persistence

Then retrieve the document and verify it exists.

Q57. How would you test persistence?

Answer:

A strong integration test is:

POST

↓

document created

GET by ID

↓

same document returned

This proves that the data actually survived the boundary between the API and database.

Q58. How do you test update?

Answer:

Test:

POST

↓

PUT

↓

GET

Then verify that the returned document contains the updated values.

Q59. How do you test delete?

Answer:

Test:

POST

↓

DELETE

↓

GET

The final GET should produce:

404

Q60. What is test isolation?

Answer:

Test isolation means one test should not unexpectedly depend on data created by another test.

Bad:

Test A creates doc-001

↓

Test B assumes doc-001 exists

If Test A doesn't execute first, Test B fails.

Good:

Test A → independent database state

Test B → independent database state

Q61. How can database tests achieve isolation?

Answer:

Common strategies include:

- dedicated test database
- fixture setup/teardown
- transaction rollback
- truncating test tables
- isolated schemas
- disposable database instances

For Day 5, you need to understand the problem and verify a reasonable approach rather than building sophisticated infrastructure.

Q62. Why might transaction rollback be useful in tests?

Answer:

A test can execute database operations inside a transaction and roll them back afterward.

Conceptually:

BEGIN

↓

test operations

↓

assertions

↓

ROLLBACK

This prevents test data from leaking into subsequent tests.

Q63. What is a fixture?

Answer:

A fixture provides reusable test setup or resources.

For example:

database fixture

↓

prepare database

↓

run test

↓

cleanup

Fixtures reduce duplication and make test setup consistent.

Q64. What is the difference between unit and integration testing here?

Answer:

Unit test

Tests one component in isolation.

Example:

DocumentService

↓

mock repository

Integration test

Tests multiple real components together.

Example:

API

↓

Service

↓

PostgreSQL repository

↓

PostgreSQL

For Day 5, PostgreSQL-backed integration tests are important.

PART X — CONCURRENCY AND CONSISTENCY

Q65. What happens if two requests update the same document simultaneously?

Answer:

Both requests may race to modify the same record.

The outcome depends on:

- transaction behavior
- isolation level
- locking
- update semantics
- application-level concurrency control

This is a production concern that becomes increasingly important as the service scales.

Q66. Give an example of a lost update.

Answer:

Suppose the document version is:

1.0

Two requests read it simultaneously.

Request A reads 1.0

Request B reads 1.0

A changes it to:

2.0

B changes it to:

1.5

If both blindly write their values, one update can overwrite another.

That is a lost-update problem.

Q67. How could you prevent lost updates?

Answer:

Possible strategies include:

- database row locking
- optimistic concurrency control
- version columns
- conditional updates
- transactions

For example, optimistic concurrency can conceptually use:

```
UPDATE documents
SET version = ...
WHERE id = ...
AND version = expected_version;
```

If zero rows are updated, another request may have modified the document.

PART XI — DSA: PREFIX SUM

Q68. What is Prefix Sum?

Answer:

Prefix Sum stores cumulative sums.

Given:

[2, 4, 1, 3]

we can build:

[0, 2, 6, 7, 10]

where:

`prefix[i]`

represents the sum of elements before index *i*.

Q69. Why is Prefix Sum useful?

Answer:

It converts repeated range-sum calculations from potentially:

$O(n)$

per query into:

$O(1)$

after:

$O(n)$

preprocessing.

This is the central idea behind Range Sum Query — Immutable.

Q70. What is the range-sum formula?

Answer:

Using:

`prefix[i] = sum of elements before i`

the sum from left through right is:

`prefix[right + 1] - prefix[left]`

This formula is explicitly required for Day 5.

Q71. What pattern is used for Subarray Sum Equals K?

Answer:

Prefix Sum + Hash Map

The key observation is:

`prefix[j] - prefix[i] = k`

Therefore:

`prefix[i] = prefix[j] - k`

If the current prefix sum is S , we need to know how many previous prefix sums equal:

$S - K$

This is the central insight.

Q72. Why do we use a hash map?

Answer:

Because we need fast lookup of how many times a previous prefix sum occurred.

A hash map gives expected:

$O(1)$

lookup.

Therefore the overall algorithm can be:

$O(n)$

rather than checking every possible subarray.

Q73. Why initialize the prefix-sum map with {0: 1}?

Answer:

It represents the empty prefix.

Suppose:

`current_prefix == k`

Then:

`current_prefix - k = 0`

The initial zero allows us to count a subarray starting at index 0.

This is one of the most important implementation details.

Q74. Why store frequencies rather than just whether a prefix exists?

Answer:

Because the same prefix sum can occur multiple times.

Each occurrence can correspond to a different subarray ending at the current position.

Therefore:

`prefix_sum → number of occurrences`

is required.

Q75. Does Subarray Sum Equals K require all numbers to be positive?

Answer:

No.

This is an important advantage over many Sliding Window approaches.

Prefix Sum + Hash Map works with:

positive

zero

negative

values.

Q76. Why can't a normal Sliding Window always solve Subarray Sum Equals K?

Answer:

For variable-size Sliding Window, shrinking/expanding based on whether the current sum is too large relies on useful monotonic behavior.

Negative numbers break that property.

For example:

[1, -1, 2, ...]

Adding an element can decrease the sum.

Therefore Prefix Sum + Hash Map is the more general technique.

PART XII — PRODUCT OF ARRAY EXCEPT SELF

Q77. What pattern is used?

Answer:

Prefix Product + Suffix Product

The goal is to compute the product of every element except the current element without using division.

The Day 5 target is:

$O(n)$ time

$O(1)$ extra space

excluding the output array under the stated space convention.

Q78. Why not simply multiply everything and divide by the current value?

Answer:

Because division creates problems when the input contains zero.

For example:

[1, 2, 0, 4]

Division-based solutions require special handling.

The prefix/suffix approach avoids division entirely.

Q79. Explain the prefix/suffix idea.

Answer:

For each position:

```
answer[i]
=
product of everything to the left
×
product of everything to the right
```

Example:

[1, 2, 3, 4]

For index 2:

left = 1×2

right = 4

answer = $2 \times 4 = 8$

Q80. How can you achieve $O(1)$ extra space?

Answer:

Use the output array itself to store the prefix products.

Then perform a second pass from right to left while maintaining a suffix product.

Conceptually:

Pass 1:

```
output[i] = left product
```

Pass 2:

```
output[i] *= right product
```

Thus:

Time = $O(n)$

Extra = $O(1)$

excluding the required output array.

Q81. What happens when there are zeros?

Answer:

The prefix/suffix algorithm handles zeros naturally.

For example:

[1, 2, 0, 4]

Only the position containing zero receives the product of all non-zero elements:

[0, 0, 8, 0]

With two zeros, every result becomes zero.

PART XIII — RANGE SUM QUERY — IMMUTABLE

Q82. What is the core idea?

Answer:

Precompute prefix sums once.

Then:

```
sum(left, right)
=
prefix[right + 1] - prefix[left]
```

So:

preprocessing = $O(n)$

query = $O(1)$

This is exactly the intended Day 5 transformation.

Q83. Why is this better than summing the range every time?

Answer:

If there are many queries, repeatedly iterating over ranges is expensive.

Without prefix sums:

$Q \text{ queries} \times O(n)$

could become very expensive.

With prefix sums:

$O(n)$ preprocessing

+

$Q \times O(1)$

This is valuable when queries significantly outnumber updates.

Q84. When would Prefix Sum be inappropriate?

Answer:

If the array changes frequently, maintaining prefix sums can become expensive because updates may require modifying many cumulative values.

For dynamic range queries, structures such as:

- Fenwick Trees
- Segment Trees

may be more appropriate.

Those are beyond today's Day 5 scope but are good interview follow-ups.

PART XIV — PREFIX SUM VS SLIDING WINDOW

Q85. When should you think "Sliding Window"?

Answer:

Look for problems involving a contiguous window where you can efficiently expand/shrink the window while maintaining a useful invariant.

Examples:

Longest substring with constraint

Minimum window satisfying condition

Fixed-size subarray statistics

Q86. When should you think "Prefix Sum"?

Answer:

Think Prefix Sum when you need:

- repeated range sums
- cumulative quantities
- subarray sum relationships
- efficient range calculations

For Subarray Sum Equals K:

Prefix Sum + Hash Map

For Range Sum Query:

Prefix Sum

Q87. What's the difference between Prefix Sum and Sliding Window?

Answer:

Technique | Main idea

Sliding Window | Maintain a moving contiguous region

Prefix Sum | Precompute cumulative information

Prefix + Hash Map | Use cumulative sums to detect subarrays

The key is pattern recognition, not memorizing names.

PART XV — PROJECT-BASED GOOGLE QUESTIONS

These are especially important because they connect theory to your flagship project.

Q88. Walk me through your Day 5 architecture.

Strong answer:

I started with a FastAPI document service using an API → service → repository architecture and in-memory persistence. On Day 5, I replaced the persistence implementation with PostgreSQL without changing the API contract or business-service boundary.

Then:

Client

↓

FastAPI Router

↓

DocumentService

↓

DocumentRepository

↓

PostgreSQL

The API handles HTTP concerns, the service handles business logic, and the repository handles persistence.

Q89. Why didn't you put SQL inside the FastAPI route?

Answer:

Because the route should be responsible for HTTP concerns rather than business and persistence logic.

If SQL were inside the route:

route

- validation
- business logic
- SQL
- database connection
- response mapping

the endpoint would become difficult to test and maintain.

The layered architecture keeps these responsibilities separated.

Q90. Why didn't you change the API when moving to PostgreSQL?

Answer:

Because persistence implementation should not leak through the API boundary.

The client shouldn't care whether:

dictionary

PostgreSQL

stores the document.

Keeping the API stable demonstrates that the repository abstraction is working.

Q91. What was the most important architectural decision on Day 5?

Answer:

Keeping:

API → Service → Repository

unchanged while replacing:

In-Memory Store

with:

PostgreSQL

This demonstrates separation of concerns and replaceable persistence.

Q92. What if the database returns no document?

Answer:

The repository should communicate that the document doesn't exist.

The service can translate that into the domain-level behavior already established.

The API's centralized exception handling then maps that condition to:

HTTP 404

The Day 5 goal is to preserve the existing exception architecture rather than redesign it.

Q93. What happens if PostgreSQL raises an unexpected error?

Answer:

The application should not return a fake successful response.

The error should be:

detected

↓

logged appropriately

↓

translated into safe HTTP behavior

while avoiding exposing internal database details to the client.

Q94. How would you handle a duplicate document ID?

Answer:

The database's primary-key constraint should reject the duplicate.

The application should catch the relevant database exception and translate it into an appropriate application/API response.

The exact status depends on the API contract.

Q95. What if the service says a document exists but PostgreSQL says it doesn't?

Answer:

That indicates an inconsistency in the application's assumptions or persistence behavior.

I would trace:

API request

↓

Service

↓

Repository

↓

SQL

↓

database result

Then inspect:

- transaction boundaries
- query conditions
- connection/database configuration
- test isolation
- concurrent modifications
- stale assumptions

This is an excellent debugging interview question.

PART XVI — SYSTEM DESIGN

Day 5 asks you to design a document storage service for 1 million documents using:

Requirements

Scale

API

Data Model

Architecture

Indexes

Caching

Partitioning

Consistency

Reliability

Monitoring

Cost

without implementing the scaled system yet.

Q96. Design a document storage service for 1 million documents.

Answer framework:

Requirements

Support:

create

read

list

update

delete

Scale

Assume:

1M documents

and determine:

read/write ratio

request rate

document metadata size

growth rate

availability target

API

POST /documents

GET /documents

GET /documents/{id}

PUT /documents/{id}

DELETE /documents/{id}

Data model

PostgreSQL:

documents

with appropriate primary key and indexes.

Architecture

Clients



Load Balancer



FastAPI instances



DocumentService



Repository



PostgreSQL

Indexes

Primary key index for ID lookup.

Additional indexes only for demonstrated query patterns.

Caching

Potentially cache frequently accessed immutable/stable metadata, but introduce caching only when the workload justifies it.

Partitioning

One million rows does not automatically mean partitioning is required.

Partition only if access patterns, data growth, maintenance, or database limits justify it.

Consistency

Start with strong consistency for document metadata unless requirements say otherwise.

Reliability

Consider:

- database backups
- replication
- connection limits
- health checks
- retries where safe
- failure handling

Monitoring

Track:

- request latency
- error rate
- database latency
- connection pool utilization
- query performance
- database storage
- CPU/memory

Cost

Avoid unnecessary replicas, indexes, caching infrastructure, or partitioning until justified by requirements.

Q97. How would you scale PostgreSQL?

Answer:

Start by measuring the actual bottleneck.

Potential progression:

Optimize queries

↓

Indexes

↓

Connection pooling

↓

Vertical scaling

↓

Read replicas

↓

Caching

↓

Partitioning/sharding if genuinely required

Do not jump directly to sharding.

Q98. When would you introduce read replicas?

Answer:

When read traffic becomes large enough that separating read workloads from the primary database provides value.

Architecture:

Writes

↓

Primary

Reads

↓

Read replicas

But this introduces replication lag and therefore consistency considerations.

Q99. What problem does caching solve?

Answer:

Caching reduces repeated expensive reads.

For example:

Client

↓

Cache

↓ miss

PostgreSQL

A cache can reduce:

- database load
- latency
- repeated computation

But it introduces:

- invalidation
- stale data
- consistency complexity
- additional infrastructure

Therefore caching should be justified by workload.

Q100. Would you partition the database with only 1 million documents?

Answer:

Not automatically.

One million rows is not by itself sufficient justification for partitioning.

I would first measure:

- query performance
- table growth
- maintenance requirements
- storage characteristics
- access patterns

Then determine whether partitioning solves an actual problem.

PART XVII — GOOGLE-STYLE SCENARIO QUESTIONS

Q101. Your API suddenly becomes slow. How do you debug it?

Answer:

I would avoid immediately assuming PostgreSQL is the problem.

Trace the request:

Client

↓

FastAPI

↓

Service

↓

Repository

↓

PostgreSQL

Measure:

- API latency
- service execution time
- database query latency
- connection acquisition time
- query plans
- database CPU
- connection pool utilization

Then identify the actual bottleneck.

Q102. PostgreSQL CPU reaches 100%. What do you investigate?

Answer:

I would investigate:

- 1. slow queries
- 2. query execution plans
- 3. missing/ineffective indexes
- 4. sudden traffic increase
- 5. expensive joins
- 6. large scans
- 7. concurrent transactions
- 8. connection behavior

I would measure first rather than blindly adding indexes.

Q103. Tests pass individually but fail when run together. What might be wrong?

Answer:

Likely test isolation.

Possible causes:

- shared database state
- missing cleanup

- fixtures leaking state
- transaction not rolled back
- order-dependent tests

This is exactly why Day 5 introduces database test isolation.

Q104. A test passes locally but fails in CI. What would you investigate?

Answer:

I would compare:

environment

database configuration

schema/migrations

dependencies

Python version

environment variables

test ordering

database initialization

A common failure is that local PostgreSQL state exists but CI starts from an empty database.

This is why reproducible schema creation matters.

Q105. Why should database schema creation be reproducible?

Answer:

Because a developer or CI system should be able to recreate the database from a known state.

Bad:

"I manually created this table six days ago."

Good:

fresh environment

↓

migration/schema mechanism

↓

same database structure

PART XVIII — DEEP FOLLOW-UPS

Q106. Why not use MongoDB for documents?

Answer:

The term "document" here describes the domain object, not necessarily the database model.

Our current data is structured metadata with predictable fields:

id

name

type

version

status

timestamps

A relational database provides strong constraints, transactions, SQL querying and structured integrity.

If later Project 1 stores arbitrary extracted document content, a hybrid architecture could become reasonable, but Day 5 does not require introducing that complexity.

Q107. Why PostgreSQL instead of SQLite?

Answer:

SQLite is excellent for lightweight local applications and embedded use cases.

PostgreSQL is more appropriate for the trajectory of this project because we eventually care about:

- concurrent server workloads
- transactions
- indexing
- production deployment
- scalability
- operational database capabilities

However, the choice should always be driven by requirements rather than technology preference.

Q108. What is normalization?

Answer:

Normalization is organizing relational data to reduce unnecessary duplication and update anomalies.

For example, if many documents belong to the same project, instead of repeating project information in every row, we could have:

projects

documents

with:

documents.project_id

referencing:

projects.id

Day 5 does not require deep normalization design, but you should understand the concept.

Q109. What is denormalization?

Answer:

Denormalization intentionally duplicates data to improve read performance or simplify access patterns.

The trade-off is:

faster/simpler reads

vs

more duplicated data

and consistency complexity

Q110. What is an N+1 query problem?

Answer:

Suppose we retrieve:

100 documents

and then execute another database query for each document's metadata.

That produces:

1 query + 100 queries

= 101 queries

This can severely hurt performance.

A better approach may use joins, batching, or appropriate query design.

PART XIX — RAPID-FIRE REVISION

Before marking Day 5 complete, you should be able to answer these without notes:

- 1. What is PostgreSQL?
- 2. What is a relational database?
- 3. What is a table?
- 4. What is a row?
- 5. What is a column?

- 6. What is a primary key?
- 7. What is a foreign key?
- 8. What is NOT NULL?
- 9. What is UNIQUE?
- 10. What is CHECK?
- 11. What does INSERT do?
- 12. What does SELECT do?
- 13. What does WHERE do?
- 14. What does UPDATE do?
- 15. What does DELETE do?
- 16. Why parameterize SQL?
- 17. What is SQL injection?
- 18. What is a transaction?
- 19. What is COMMIT?
- 20. What is ROLLBACK?
- 21. What is ACID?
- 22. What is isolation?
- 23. What is an index?
- 24. Why do indexes improve reads?
- 25. Why do indexes hurt writes?
- 26. What is connection pooling?
- 27. Why use environment variables?
- 28. Why shouldn't .env be committed?
- 29. Why use a repository?
- 30. Why shouldn't the service execute SQL?
- 31. What happens when PostgreSQL is unavailable?
- 32. What is test isolation?
- 33. What is a database fixture?
- 34. What is transaction rollback in tests?
- 35. What is Prefix Sum?
- 36. Why initialize prefix frequency with zero?
- 37. Why does Subarray Sum Equals K use a hash map?
- 38. Why can it handle negative numbers?
- 39. Why use prefix + suffix for Product Except Self?
- 40. Why avoid division?
- 41. Why is Range Sum Query $O(1)$ per query?
- 42. When would Sliding Window be preferable?
- 43. When would Prefix Sum be preferable?
- 44. What happens when two requests update the same document?

- 45. How would you scale the document service?

PART XX — TOP 20 QUESTIONS YOU MUST MASTER

If your preparation time is limited, prioritize these:

Backend / Database

- 1. Why PostgreSQL for this project?
- 2. Explain your API → Service → Repository → PostgreSQL architecture.
- 3. Why use a repository abstraction?
- 4. Why shouldn't business logic execute SQL directly?
- 5. Explain primary keys and why document_id needs one.
- 6. Explain database constraints and why they matter.
- 7. Explain transactions with a real failure example.
- 8. Explain indexes and their read/write trade-off.
- 9. How do you manage PostgreSQL connections?
- 10. How do you handle database failure?

Testing

- 11. How do you test PostgreSQL-backed APIs?
- 12. What is test isolation?
- 13. How would you prevent test data from leaking between tests?

DSA

- 14. Explain Prefix Sum from first principles.
- 15. Derive $\text{prefix}[\text{right} + 1] - \text{prefix}[\text{left}]$.
- 16. Explain Subarray Sum Equals K.
- 17. Why do we store prefix-sum frequencies?
- 18. Explain Product of Array Except Self without division.

System Design

- 19. Design document storage for 1 million documents.
- 20. Two requests update the same document simultaneously. How do you prevent lost updates?

Day 5 Interview Standard

Do not consider yourself interview-ready merely because you can recite these answers.

Your real test is:

Can you explain the answer, connect it to your implementation, defend the design decision, and handle a follow-up that changes the assumptions?

For example, don't stop at:

"Indexes make queries faster."

You should be able to continue:

"They improve reads by providing an auxiliary lookup structure, but they consume storage and add maintenance work to inserts and updates. Therefore I would index based on actual query patterns rather than indexing every column."

That is the level we are targeting for Google.

The Day 5 plan's Definition of Done also requires that the actual implementation matches the documentation, PostgreSQL-backed CRUD works, missing documents still produce 404, tests are isolated/considered, and the three DSA problems have their patterns, complexity and edge cases understood.

Next dependency: once you can answer these questions and your PostgreSQL implementation passes its tests, Day 6 should build forward from persistent backend + Prefix Sum rather than repeating Day 5.